4. Functional programming part (Scheme) [8% of your final mark]

Create a Scheme function that solves the stable matching problem for residents and programs. Since the functional programming paradigm will be used here, it is a natural choice to use the recursive McVitie-Wilson algorithm.

To help you with your design, we provide you with the two Scheme functions that reads the csv files to produce the lists of residents and programs.

```
> (read-residents "residentSmall.csv")
((574 "Salvatore" "Williams" ("NRS" "HEP" "MMI"))
 (517 "Rosalie" "Frederick" ("NRS" "MMI"))
 (403 "Aspyn" "Olson" ("HEP" "NRS"))
 (226 "Zev" "Jarvis" ("MMI" "HEP" "NRS"))
 (126 "Indie" "Medrano" ("MMI" "NRS"))
 (828 "Emma" "Tremmo" ("OBG" "NRS"))
 (913 "Camille" "Paquet" ("OBG" "MMI" "HEP"))
 (773 "Marie" "Clown" ("OBG"))
 (616 "Laurent" "Robert" ("OBG" "MMI" "HEP" "NRS"))
 (377 "Tom" "Tan" ("MMI" "OBG")))
> (read-programs "programSmall.csv")
(("MMI" "Microbiology" 1 (574 517 226 913 377 126))
 ("HEP" "Hematological Pathology" 2 (403 574 913 616 226))
 ("NRS" "Neurosurgery" 4 (574 517 403 828 226 126))
 ("OBG" "Obstetrics and Gynecology" 3 (616 828 773 913)))
```

These two lists will constitute the input to your program. However, instead of using the return value of these function calls directly in your program, we ask you to define the two names `PLIST` and `RLIST`, associating them with the two lists.

```
(define PLIST (read-programs "programSmall.csv"))
(define RLIST (read-residents "residentSmall.csv"))
```

With these definitions, the stable matches can be obtained by this function call:

```
> (gale-shapley RLIST PLIST '())
(("OBG" ((773 . 2) (828 . 1) (616 . 0))) ("MMI" ((226 . 2))) ("HEP"
((913 . 2) (403 . 0))) ("NRS" ((126 . 5) (517 . 1) (574 . 0))))
```

The output is a list of matches. Each element of this list is a sub list made of a program ID and a list of pairs made of the resident ID and the rank of this resident in the ROL of the program. The reason for adding the rank information to the solution is strictly for optimization; this is to avoid re-computing rank positions multiple times as this could quickly becomes computationally prohibitive for ROLs containing thousands of candidate residents.

As always, to successfully develop your solution, the best approach is to design several short low-level functions that will be assembled together to build to the complete solution. Here is a list of the functions you should define:

- `(get-resident-info rid rlist)`: that returns the information associated with a given resident in the list of residents.

  ```
  > (get-resident-info 574 RLIST)
  (574 "Salvatore" "Williams" ("NRS" "HEP" "MMI"))
  ```

- `(get-program-info pid plist)`: that returns the information associated with a given program in the list of programs.

  ```
  > (get-program-info "HEP" PLIST)
  ("HEP" "Hematological Pathology" 2 (403 574 913 616 226))
  ```

- `(rank rid pinfo)`: that returns the rank of a resident in the ROL of a program (here 0-based).

  ```
  > (rank 616 (get-program-info "HEP" PLIST))
  3
  ```

- `(matched? rid matches)`: that returns true if this resident has been matched in one of the programs of the list of matches.

  ```
  > (matched? 403 (gale-shapely RLIST PLIST '()))
  #t
  > (matched? 377 (gale-shapely RLIST PLIST '()))
  #f
  ```

- `(define (get-match pid matches)`: that returns the match information associated with a given program from the list of matches.

```
> (get-match "HEP" (gale-shapely RLIST PLIST '()))
("HEP" ((913 . 2) (403 . 0)))
> (get-match "NRS" (gale-shapely RLIST PLIST '()))
("NRS" ((126 . 5) (517 . 1) (574 . 0)))
```

- `(add-resident-to-match pair match)`: that add a resident (and its rank) to the resident match list of the program.

```
> (add-resident-to-match (cons 828 3)
     (get-match "NRS" (gale-shapely RLIST PLIST '())))
("NRS" ((126 . 5) (828 . 3) (517 . 1) (574 . 0)))
```

Notice how, in the case of the last function, the residents are always added in the reverse order of their ranks. This way, the least preferred resident is always the first element in the list of matched residents. This is important since, when a program has reached its quota, you need to replace that least preferred resident by a new one if this new resident is preferred over this currently least preferred resident. It is therefore very convenient to have this resident first in the list since you often need to access it, and this is done easily using the `car` function.

The McVitie-Wilson functions

Since we are using the McVitie-Wilson algorithm, you will need to implement the `offer` and `evaluate` functions. For the `offer` function, it is useful to input the list of programs and residents in addition to the resident information. A current list of matches is also inputted to the function which then returns an updated list.

```
(offer rinfo rlist plist matches)

> (offer (car RLIST) RLIST PLIST '())
(("NRS" ((574 . 0))))
> (offer (cadr RLIST) RLIST PLIST
     (offer (car RLIST) RLIST PLIST '()))
(("NRS" ((517 . 1) (574 . 0))))
> (offer (caddr RLIST) RLIST PLIST
     (offer (cadr RLIST) RLIST PLIST
          (offer (car RLIST) RLIST PLIST '())))
(("HEP" ((403 . 0))) ("NRS" ((517 . 1) (574 . 0))))
```

Of course, if the resident cannot be matched (no program can be offered), then the list of matches returned is the same as the one specified as input. You follow a similar pattern for the `evaluate` function except that this time you also input the program information.

```
(evaluate rinfo pinfo rlist plist matches)

> (define M1 (offer (caddr RLIST) RLIST PLIST
     (offer (cadr RLIST) RLIST PLIST
          (offer (car RLIST) RLIST PLIST '()))))
> (define M2 (evaluate (get-resident-info 226 RLIST)
               (get-program-info "HEP" PLIST) RLIST PLIST M1))
> M2
(("HEP" ((226 . 4) (403 . 0))) ("NRS" ((517 . 1) (574 . 0))))

> (evaluate (get-resident-info 913 RLIST) (get-program-info "HEP"
PLIST) RLIST PLIST M2)
(("MMI" ((226 . 2))) ("HEP" ((913 . 2) (403 . 0)))
                        ("NRS" ((517 . 1) (574 . 0))))
```

Remember that the `evaluate` function also contains a call to `offer` that is invoked whenever a less preferred resident is kicked out of a program that reached its quota.

As per the McVitie-Wilson algorithm, the solution to the stable matching problem is found by calling the `offer` function for all the residents in the list read from the csv file. This is accomplished by the `gale-shapley` function as explained above. This one produces a list of matches. However, the final requirement is the same than for the other parts of this project. You therefore need to create a top-level function that will print the resuls from the list provided by the `gale-shapley` function; this function should be named `gale-shapley-print`. In Lisp, the printing functions are generally the only ones containing sequences (instead of strict function compositions). To guide you in your design, we provide you here with an example showing how this function could look like:

```
(define (gale-shapley-print rlist plist)
     (let* ((matches (gale-shapley rlist plist '()))
            (not-matched-list (get-not-matched-list rlist matches)))
          (for-each (lambda(m)
               (display-program-matches m rlist plist)) matches)
          (display-not-matched not-matched-list rlist)
          (display "Number of unmatched residents: ")
               (display (length not-matched-list)) (newline)
          (display "Number of positions available: ")
          (display (get-total-available-positions matches plist))
          (newline)))
```

```
> (gale-shapley-print RLIST PLIST)
Clown,Marie,773,OBG,Obstetrics and Gynecology
Tremmo,Emma,828,OBG,Obstetrics and Gynecology
Robert,Laurent,616,OBG,Obstetrics and Gynecology
Jarvis,Zev,226,MMI,Microbiology
Paquet,Camille,913,HEP,Hematological Pathology
Olson,Aspyn,403,HEP,Hematological Pathology
Medrano,Indie,126,NRS,Neurosurgery
Frederick,Rosalie,517,NRS,Neurosurgery
Williams,Salvatore,574,NRS,Neurosurgery
Tan,Tom,377,XXX,NOT_MATCHED
Number of unmatched residents: 1
Number of positions available: 1
```

Remember that Scheme and its functional paradigm are not always computationally efficient. If your program takes too long to execute on the 4000-resident dataset, try to identify the bottleneck functions. For this, the time function is very useful, this one displays in ms, the elapsed time to evaluate a function. For example:

```
> (time (gale-shapely RLIST PLIST '()))
cpu time: 117140 real time: 121489 gc time: 11546
(("MMI"
  ((72335487 . 288)
   (51837808 . 287)
   (93640833 . 286)
     …
```

And here is what is displayed when all calls to `offer` are enclosed in a `time` function (here we show only the 5 last calls):

```
> (time (gale-shapely RLIST PLIST '()))
…
cpu time: 0 real time: 6 gc time: 0
cpu time: 15 real time: 13 gc time: 0
cpu time: 15 real time: 10 gc time: 0
cpu time: 0 real time: 6 gc time: 0
cpu time: 15 real time: 9 gc time: 0
…
```

Submission

Your solution must include the `gale-shapley` and `gale-shapley-print` top-level functions and all the enumerated functions. Submit your project in a zip file containing the scheme functions; all your functions must have a short header describing what the function does. Include also a text file (`output.txt`) containing the results produced for the `residents4000.csv` and `programs4000.csv` files.

Your function must adhere to the functional paradigm principles. In particular you must not use the functions terminating by `!` (such as the `set!` function or the Scheme hash table data structure).